Faculty of Engineering & Technology

Electrical & Computer Engineering Department

Operating Systems – ENCS3390

Project Report

Prepared by :

Name : Sara Shrouf Number: 1220481

Instructor: Abdel Salam Sayyad

Date: 7/12/2024 .

Table of Contents

Discussion and Analysis : 5

Describe my environment : 5

Analysis : 5

Naïve code : 6

Analysis : 7

Multiprocessing approach : 8

Analysis : 8

Multithreading approach: 9

Analysis : 9

Performance Measurements : 11

Performance 11

Execution Time 11

Project Questions : 12

Conclusion : 13

Table of Figure:

Figure 1 :vm setting…… 2

Figure 2: task maneger ……3

Figure 3: struct code …… 4

Figure 4: read data……5

Figure 5: naïve code……6

Figure 6: result of multiproscess ……6

Figure 7: threadresult…… 9

List of Table

Table 1 Multiprocessing Table…….…….…….

Discussion and Analysis :

Describe my environment :

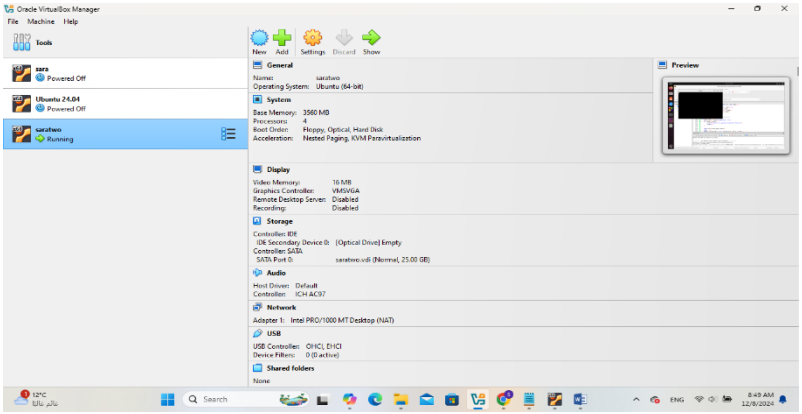

Figure 1 :vm setting

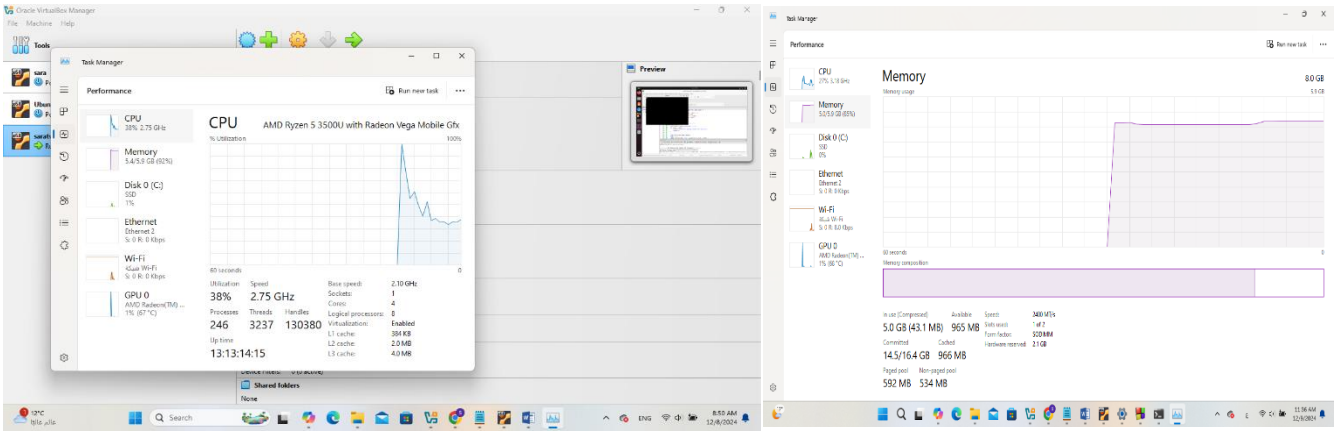

Figure 2 : task maneger

Analysis :

The system is equipped with an AMD Ryzen 5 3500U processor, featuring 4 cores and 8 threads. The base clock speed is 2.1 GHz, and it can boost dynamically based on workload. This processor is well-suited for both virtualization and multithreaded programming tasks also The system has 8 GB of RAM installed. During the project, approximately 54% of the RAM was utilized, leaving sufficient memory for both the host operating system and the virtual machine. For development and execution, I created a virtual machine running Ubuntu 18.04 Linux. This Linux distribution is lightweight and compatible with the required C development tools

The virtual machine was configured with the following resources :

Memory : 2.5 GB of RAM

Cores : 4 cores were allocated to meet the project's requirements for parallel processing and thread execution

The project was implemented in the C programming language, and the Tools and IDEs is Codeblocks configured with GCC for a more graphical development experience

Naïve code :

```c

#define MAX_WORDS 17005207
#define MAX_WORD_LENGTH 100

typedef struct {
    char word[MAX_WORD_LENGTH];
    int frequency;
} Word;
```

Figure 3 : struct code

I Defined The Word structure stores each unique word and its frequency .

```c
// Function to read words from the file and store them in the word array
int read_file(Word *wordArray, int *wordCount) {
    FILE *data = fopen("text8.txt", "r");
    if (data == NULL) {
        printf("\nAn Error in opening 'text8.txt' file.\n");
        printf("\nInput file name must be (text8.txt).\n");
        return 1;
    }

    char buffer[MAX_WORD_LENGTH];
    while (fscanf(data, "%s", buffer) != EOF) {
        store_word(wordArray, wordCount, buffer);
    }

    fclose(data);
    return 1;
}
```

Figure 4 : read data

The code reads the text file using fscanf to extract each word. It then calls store_word to update the word frequency.

```c
// Function to store all words in the array
int store_word(Word *wordArray, int *size, const char *word) {
    for (int i = 0; i < *size; i++) {
        if (strcmp(wordArray[i].word, word) == 0) {
            wordArray[i].frequency++;
            return 1;
        }
    }

    strcpy(wordArray[*size].word, word);
    wordArray[*size].frequency = 1;
    (*size)++;
    return 1;
}
```

```c
// Merge function
void merge(Word *wordArray, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    Word *L = (Word *)malloc(n1 * sizeof(Word));
    Word *R = (Word *)malloc(n2 * sizeof(Word));

    for (int i = 0; i < n1; i++)
        L[i] = wordArray[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = wordArray[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i].frequency >= R[j].frequency) {
            wordArray[k] = L[i];
            i++;
        } else {
            wordArray[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        wordArray[k] = L[i];
        i++;
        k++;
    }
```

```c
    while (j < n2) {
        wordArray[k] = R[j];
        j++;
        k++;
    }

    free(L);
    free(R);
}

// Merge Sort function
void merge_sort(Word *wordArray, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        merge_sort(wordArray, left, mid);
        merge_sort(wordArray, mid + 1, right);

        merge(wordArray, left, mid, right);
    }
}
```

```c
int main() {
    Word *wordArray = malloc(MAX_WORDS * sizeof(Word));
    if (wordArray == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    int wordCount = 0;

    clock_t start = clock();

    read_file(wordArray, &wordCount);

    merge_sort(wordArray, 0, wordCount - 1);

    clock_t end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Top 10 most frequent words:\n");
    for (int i = 0; i < 10 && i < wordCount; i++) {
        printf("%s: %d\n", wordArray[i].word, wordArray[i].frequency);
    }

    printf("\nExecution Time: %.2f seconds\n", time_taken);

    free(wordArray);

    return 0;
}
```

Figure 5 :naïve code

Analysis :

In this code i store_word in function to checks if a word already exists in the array. If it does, it increments the frequency; otherwise, it adds the word to the array and sets its frequency to 1و. This results in a time complexity of O(n) for each word insertion, where n is the number of words already stored. For a dataset of 17 million words, this becomes inefficient as the number of comparisons grows.

The overall time complexity of word storage is approximately $O(n^2)$, where n is the total number of words in the dataset. The merge_sort function is used to sort the words by frequency in descending order. The merge sort algorithm has a time complexity of O(n log n). After sorting the words by frequency, the top 10 most frequent words are printed, followed by the execution time .

Multiprocessing approach :

Analysis :

1. Multiprocessing Implementation : The program achieves multiprocessing by using the fork() system call to create multiple child processes. Each child process handles a distinct chunk of the dataset, ensuring parallel execution. The dataset is divided into equally sized chunks based on the number of processes, with each child tasked with counting the frequencies of words in its assigned portion. The parent process waits for all child processes to complete using the wait() function, ensuring synchronization before aggregating results .

2. Shared Memory Management : To allow child processes to share their results, the program uses mmap() to allocate a shared memory region. This shared memory, sharedResult, is accessible by all child processes and the parent process. Each child process updates this memory with its intermediate word frequency counts. Another shared memory variable, resultSize, tracks the number of unique words added to the shared memory. This approach eliminates the need for complex inter-process communication mechanisms, simplifying the aggregation of results.

3. Word Frequency Counting : Each child process processes its chunk of the dataset by storing unique words and their frequencies in a local array. This is achieved through the store_word() function, which checks if a word already exists in the array. If it does, the frequency is incremented; otherwise, the word is added to the array with an initial frequency of one. Once processed, the results are merged into the shared memory using the merge_results() function, which ensures that duplicate words from different chunks are combined with their frequencies summed

• Performance (Throughput) = $\mathbf{1} / \boldsymbol{Tim}$

Table 1: multiprocesses result

Number of processes :	Execution Time :	Performance
2 processes	1394.287276	0.00072
4 processes	968.236371	0.0010
6 processes	797.399819	0.0012
8 processes	787.428637	0.0012

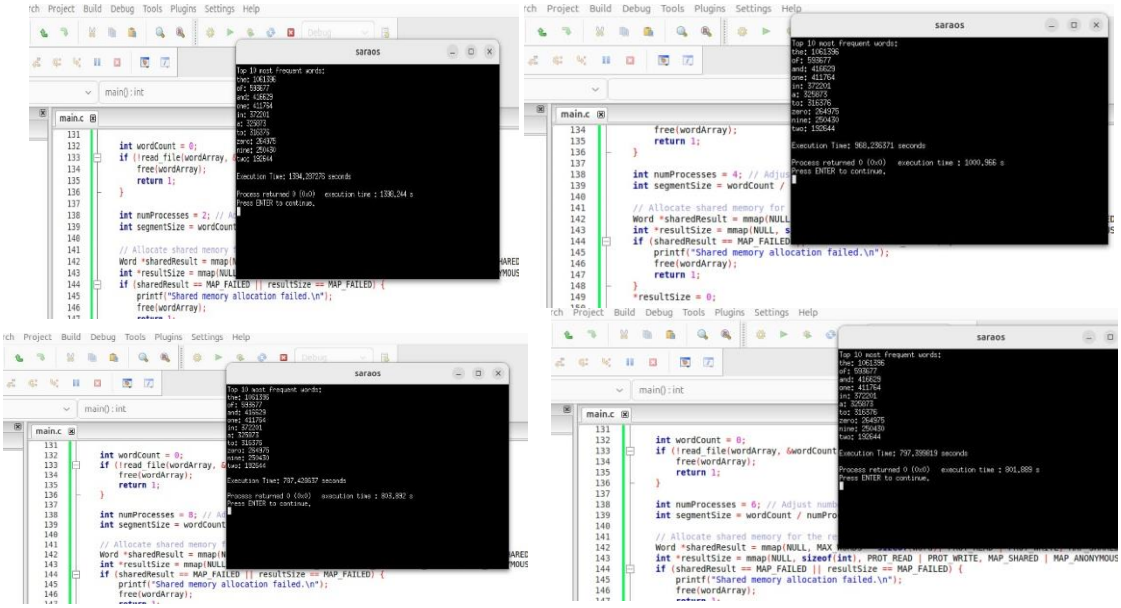

Figure 6: result of multiproscess

Increasing the number of processes from 2 to 8 reduces execution time by improving CPU utilization. At 2 processes, execution takes 1395.28 seconds, while 4 processes reduce it significantly to 968.23 seconds. With 6 processes, the time drops further to 797.39 seconds, but at 8 processes, the reduction to 787.42 seconds is minimal, showing diminishing returns due to overheads like context switching. The optimal process count is 6, balancing parallelism and overhead.

Multithreading approach:

Analysis :

in this code , he program uses multithreading to break down the task of processing a large dataset (the words in the file) into smaller, parallelizable tasks. By dividing the dataset into chunks, each thread processes a subset of the words, resulting in a significant performance improvement. The use of pthread_create() to launch threads, and e ThreadData structure is passed to each thread, which contains the startIdx and endIdx, allowing each thread to handle a specific chunk of the data.

to ensure thread-safe access to shared data, using pthread_mutex_lock(), and When a thread locks a mutex it is granted exclusive access to the shared resource Other threads that also need to modify the shared resource must wait for the mutex to be unlocked.

This waiting period introduces idle time for other threads, meaning they cannot continue processing until the mutex is released. As a result, the parallel execution benefits of multithreading are diminished because threads are essentially "paused" while waiting for access to the critical section

and pthread_join() to synchronize thread completion allows the program to handle large datasets efficiently without the risk of data corruption.

Table 2: multithread result

Number of threads	Execution Time	Performance
2 threads	934.478329	0.00107
4 threads	816.519723	0.001224
6 threads	774.511702	0.0012911
8 threads	682.890160	0.001464

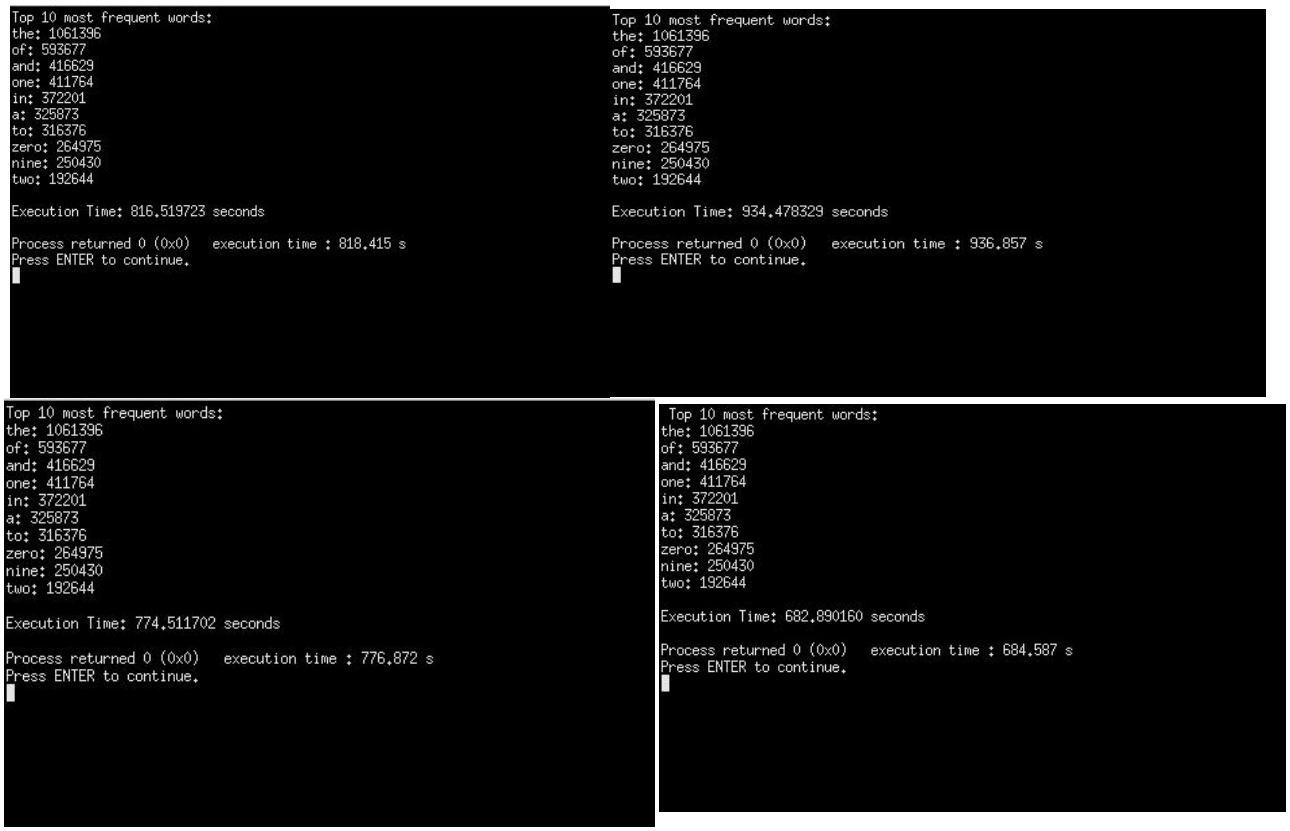

Figure 7 : threadresult

Performance Measurements :

Table 3 : Comparing Performance

	Execution Time	Performance
Naïve Approach	1799.14	0.000555
2 processes	1394.287276	0.000717212
4 processes	968.236371	0.001032805
6 processes	797.399819	0.00125407
8 processes	787.428637	0.00126995
2 threads	934.478329	0.00107
4 threads	816.519723	0.001224
6 threads	774.511702	0.0012911
8 threads	682.890160	0.001464

the naive approach, the program runs on a single process (no multiprocessing or multithreading). It takes the longest execution time (2023.86 seconds), and as expected, the performance is lower (0.000494), since the task is performed sequentially. The multiprocessing approach achieves significant improvements in execution time as the number of processes increases, from 1394.29 seconds with 2 processes to 787.43 seconds with 8 processes. This is expected, as multiprocessing allows tasks to run concurrently across multiple CPU cores, reducing the time taken for each part of the data. The performance increases with the number of processes because parallel execution improves CPU resource utilization.

However, there are diminishing returns as the number of processes increases. While the reduction in execution time from 2 to 4 processes is large, the improvement from 6 to 8 processes is relatively small. This is a typical sign of diminishing returns in parallel computing, where additional processes have less of an impact due to resource contention or the limits of parallelism in the task.

The multithreading approach also shows improvements in execution time and performance as the number of threads increases. However, the performance gain here is less dramatic than with multiprocessing. This is because threads share the same address space, meaning there is more contention for resources (like CPU and memory), which limits the amount of parallelism that can be achieved compared to separate processes.

While multithreading improves performance, it is more limited by A mutex is a synchronization primitive used to prevent multiple threads from accessing a shared resource simultaneously. While this ensures data integrity, it can also reduce the efficiency of multithreading due to contention**.**

Project Questions :

Serial part in my code :

read_file

sort_words(merge_sort)

```
Run 1:
  Serial Part Execution Time: 1958.85 seconds
  Total Execution Time: 0.32 seconds
```

Serial Portion = $(\mathbf{Serial\ Time}\ /\boldsymbol{Total\ Time}) \times \mathbf{100}\%$ =1958.85−0.32/1958.85≈0.99983

if you use 2 cores:

Maximum Speed up = $\mathbf{1}/(\boldsymbol{S} + (\mathbf{1}-\boldsymbol{S}) * \boldsymbol{N})=$

Smax=1/((1−0.99983)+(0.99983/2))=1/(0.00017+0.49992) =2.00033

Conclusion :

This project compared the performance of three approaches—naïve, multiprocessing, and multithreading—for finding the top 10 most frequent words in a dataset. The naïve approach, while simple, had the slowest execution time because it processed everything sequentially. Multiprocessing showed significant improvements by utilizing multiple CPU cores effectively, with the best results achieved using 8 processes. Multithreading also improved execution time but was less efficient than multiprocessing. Overall, multiprocessing with 8 processes proved to be the fastest and most effective method. Future enhancements could include optimizing memory usage, implementing dynamic task allocation, or exploring hybrid models that combine multiprocessing and multithreading for better performance .